



Compiling C applications

Basics



Today's agenda

- What is compiling?
- C++ as a language (shortly)
- The steps of compiling a language
- The linking process
- Environment variables for linking
- GNU Make and other tools

What is compiling?

- Process of creating machine-executable code from written code
- Goes through a number of steps to create an **object file** that can be executed by the machine
 - Executable will be in the form of "10110001..."

Compiled vs. Interpreted languages

- Not all languages need to go through the same compilation process
 - Some are interpreted during runtime
 - Compiled codes are translated directly to machine code
- Compiled languages:
 - C, C++, Fortran, ..
- Interpreted languages:
 - Python, Javascript, ..
- Even interpreted languages go through their own compilation process



The C++ Programming language

- A compiled, "high-level" language
- Can be very efficient and potent in skilled hands
- C and C++ are disproportionately popular in scientific computing
 - Highly optimized math libraries
 - Great library support for parallel programming (GPU's and multicore CPU's)

Let's try compiling

- Log in to Mahti and follow along
- The training materials can be found in our training GitHub under "Training/o2-compiling_and_parallelism"

Compiling an example program

- Open the "compiling" folder
- Take a look at the source code "helloworld.cpp"
- Let's compile the program with the GNU Compiler collection (GCC)
 - Fortran: gfortran
 - C: gcc
 - C++: g++

Note: Compiling a parallel code

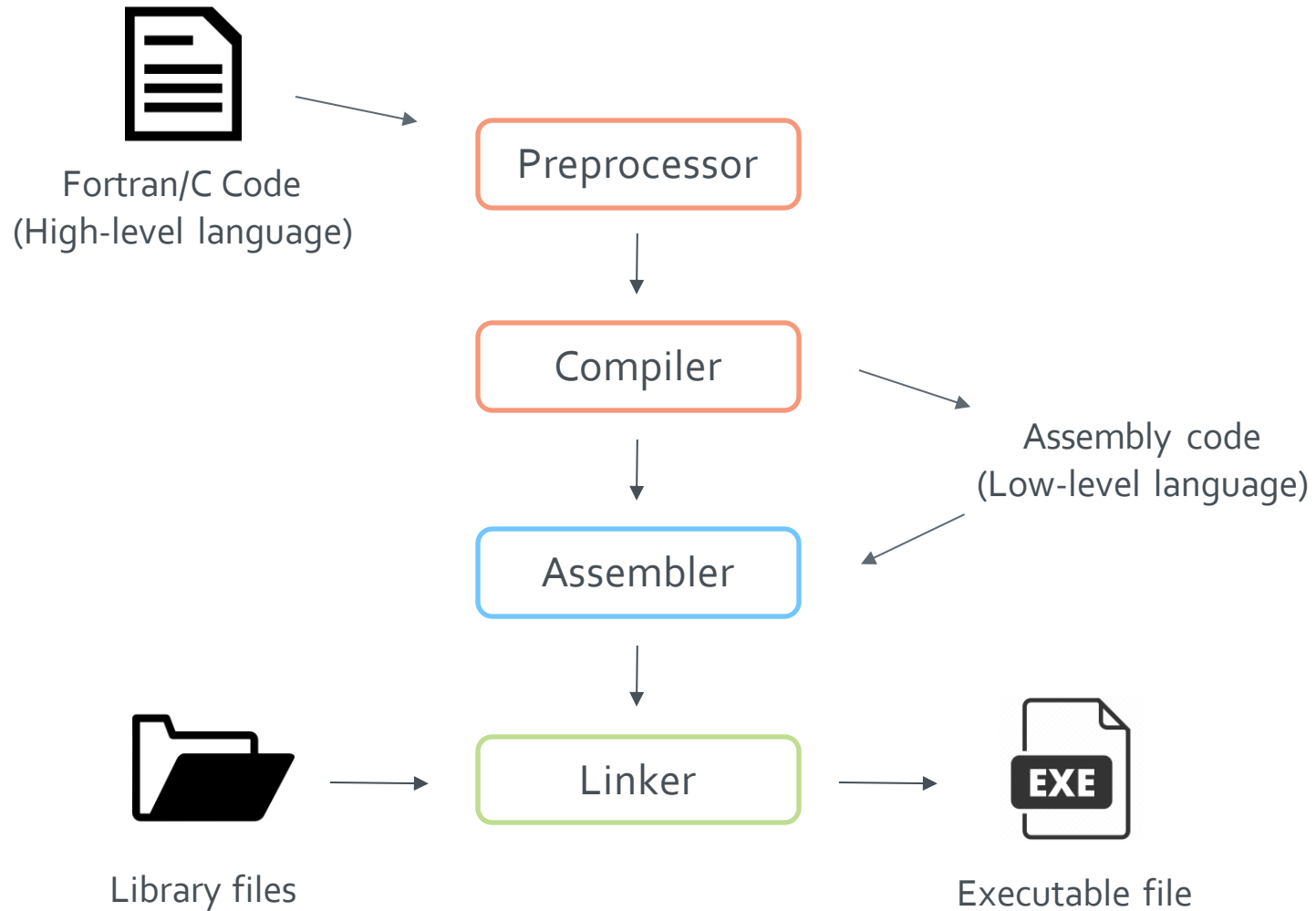
- Later we will take a look into parallel programming
- These programs often require a special compiler or specific compilation options
- In the GNU Compiler collection (GCC) when using MPI:
 - Fortran: mpifort / mpifgo
 - C: mpicc
 - C++: mpicxx / mpic++



The compilation process



Stages in compilation



o. The source code

- Fortran/C code
- Might include variables, macros, pragmas etc.
- All of this needs to be expressed in machine code before execution



Fortran/C Code
(High-level language)

```

1  #include <stdio>
2  #include <mpi.h>
3  #include <omp.h>
4
5  int main(int argc, char *argv[])
6  {
7      int my_id, omp_rank;
8      int provided, required=MPI_THREAD_FUNNELED;
9
10     MPI_Init_thread(&argc, &argv, required, &provided);
11     MPI_Comm_rank(MPI_COMM_WORLD, &my_id);
12
13     #pragma omp parallel private(omp_rank)
14     {
15         omp_rank = omp_get_thread_num();
16         printf("I'm thread %d in process %d\n", omp_rank, my_id);
17     }
18
19     if (my_id == 0) {
20         printf("\nProvided thread support level: %d\n", provided);
21         printf("  %d - MPI_THREAD_SINGLE\n", MPI_THREAD_SINGLE);
22         printf("  %d - MPI_THREAD_FUNNELED\n", MPI_THREAD_FUNNELED);
23         printf("  %d - MPI_THREAD_SERIALIZED\n", MPI_THREAD_SERIALIZED);
24         printf("  %d - MPI_THREAD_MULTIPLE\n", MPI_THREAD_MULTIPLE);
25     }
26     MPI_Finalize();
27     return 0;
28 }
29

```

1. Preprocessor

- Handles e.g. Preprocessor directives, header files, pragmas etc.
 - Expands them into the code
- Readies the code for the next step: compilation
- Creates an "expanded source" version of the original c program



Fortran/C Code
(High-level language)



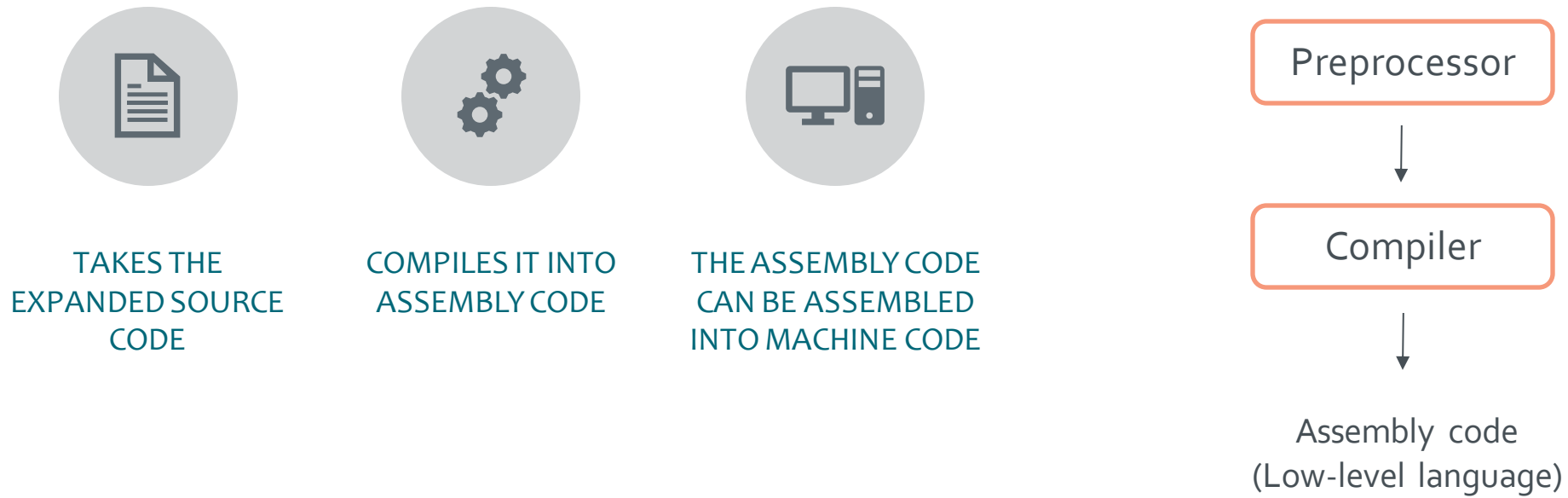
Preprocessor

```

1  #include <stdio>
2  #include <mpi.h>
3  #include <omp.h>
4
5  int main(int argc, char *argv[])
6  {
7      int my_id, omp_rank;
8      int provided, required=MPI_THREAD_FUNNELED;
9
10     MPI_Init_thread(&argc, &argv, required, &provided);
11     MPI_Comm_rank(MPI_COMM_WORLD, &my_id);
12
13     #pragma omp parallel private(omp_rank)
14     {
15         omp_rank = omp_get_thread_num();
16         printf("I'm thread %d in process %d\n", omp_rank, my_id);
17     }
18
19     if (my_id == 0) {
20         printf("\nProvided thread support level: %d\n", provided);
21         printf(" %d - MPI_THREAD_SINGLE\n", MPI_THREAD_SINGLE);
22         printf(" %d - MPI_THREAD_FUNNELED\n", MPI_THREAD_FUNNELED);
23         printf(" %d - MPI_THREAD_SERIALIZED\n", MPI_THREAD_SERIALIZED);
24         printf(" %d - MPI_THREAD_MULTIPLE\n", MPI_THREAD_MULTIPLE);
25     }
26     MPI_Finalize();
27     return 0;
28 }
29

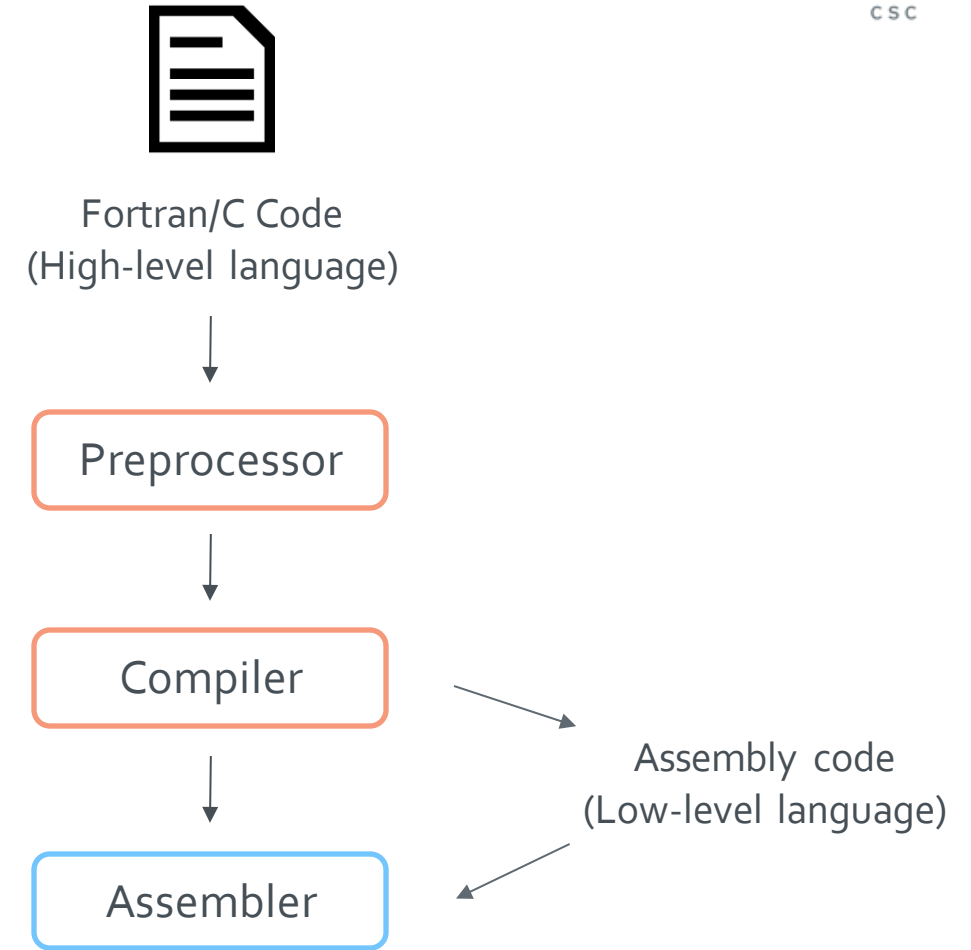
```

2. Compiler



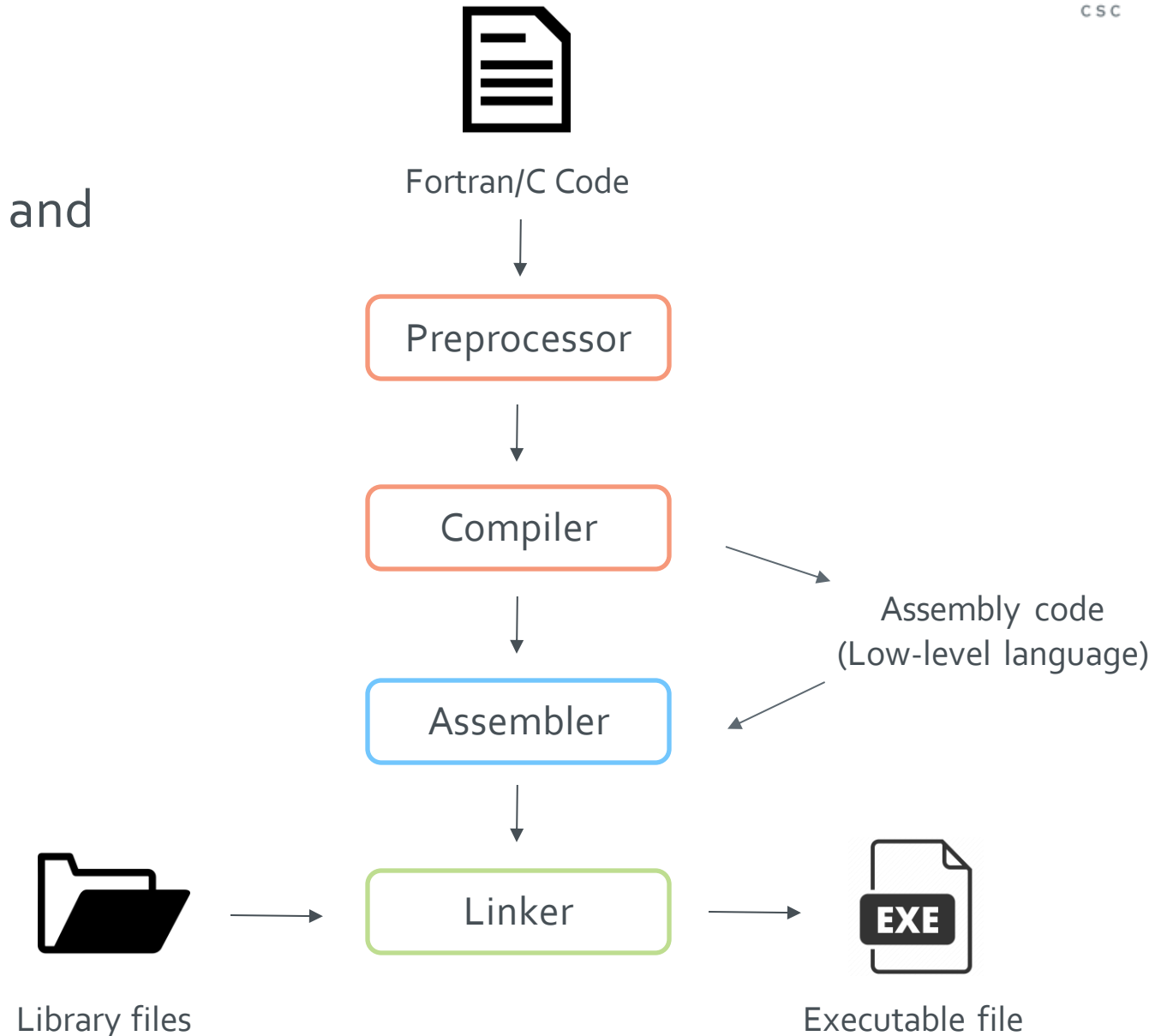
3. Assembler

- Takes the assembly code
- Creates machine-readable code from the assembly code
 - This is called an "object file"
- Almost ready at this point!



4. Linker

- Links all the needed library functions and object files together
- Here we need to focus during SCC
 - What libraries are we linking?
 - Where do the libraries reside?
 - Which MPI library are we linking?



Linking

- The user can express during compilation what libraries to link
 - During compilation with the "-l <library name>" flag
- Where to find them in the linux filesystem
 - During compilation with "-L <library_path>" flag
 - Or with an environment variable like "LD_LIBRARY_PATH"
- And what headers to include
 - During compilation with the "-I <header file>" flag
 - Or with environment variables

Linking example #1

1. Go to your local version of the competition github in Mahti
2. Download the latest main branch with "git pull"
3. Go to exercises/linking
4. Try compiling the code there straight away with g++ and see what errors you get
5. Which library are you missing and how could you load it into your environment?
 - Use "module spider <module name>" to look for the library module

Makefiles

- Compiling small codes is simple
- With larger codes the compilation process can become complicated though
 - Can also be time consuming
 - If we change single files, couldn't we just compile those?
- Having different options when building
 - Debug versions, enabling/disabling MPI etc.

Make

- Make allows the user to define how their program is compiled
 - Different versions
 - Building parts at a time
- Make rules define how some part of your program is built
- Rules are defined in a file which is by default called Makefile

Simple Makefile example

- Dependencies can be files or other targets
- Make rules define how some part of your program is built
- Rules are defined in a file which is by default called Makefile

```
CC=mpicc
OMPFLAGS=-fopenmp
LIBS=-lm

all: hello hello-omp

hello: parallel-hello.c
    mpicc -o $@ $< $(LIBS)

hello-omp: parallel-hello.c
    mpicc -o $@ $< $(LIBS) $(OMPFLAGS)

clean:
    rm -f hello hello-omp
```

Usual make commands

- Example make commands:
- Make "all" rules in the Makefile

```
make (all)
```

- Make a specific rule

```
make <rule>
```

- Clean everything that has been compiled before

```
make clean
```

Build generators

- Makefiles are not necessarily portable
- To improve portability and make dependency handling easier, build generators are often used
 - Can select the correct compilers and compiler options automatically
 - Creating Makefiles from a simpler template
- GNU Autotools and cmake are the most common build generators encountered in HPC



Makefile exercise

1. Go to your local version of the competition github in Mahti
2. Go to exercises/using_make
3. Read through the Makefile
4. Make the "hello" version to compile the MPI enabled aspects of the code



Questions?



facebook.com/CSCfi



twitter.com/CSCfi



linkedin.com/company/csc--it-center-for-science



github.com/CSCfi